

Growing Up Online: A History of the Internet in the United States (2007–2026)

Coursework report — website code submitted separately as a zip file

A. Background research

I chose the United States, framed around my own lifetime: born in 2007, the same year the first iPhone went on sale, up to the present day (2026). Rather than research the entire history of the US internet, I scoped the project to four eras that each represent a distinct kind of change I could trace through my own lifetime: home broadband and the first social platforms (2006–2010), the smartphone takeover (2007–2012), streaming and cloud computing through the COVID-19 pandemic (2006–2021), and the recent arrival of generative AI (2022–2026). For each era I researched a small number of well-documented, verifiable events — for example, home broadband adoption crossing 42% then 68% of US adults (Pew Research, NTIA), the iPhone's 2007 launch and the App Store's 2008 opening (History Cooperative, Cult of Mac), Netflix's move from DVDs to streaming and its eventual seven-year migration to AWS (VdoCipher, Silicon UK), and ChatGPT's unprecedented adoption curve after its November 2022 release (History.com, Harvard Gazette). Every fact was checked against a named, dated source before being written into the site, and every source is linked from the site's own Sources page rather than only cited in this report, so a reader can verify each claim directly from the live pages, not just from this document.

B. Planning

I planned the project as a five-stage gated pipeline matching the assignment's own structure: research and content planning first, then wireframes, then the coded build, then testing, then this report. Each stage was treated as a prerequisite for the next rather than run in parallel, because the biggest risk on a one-person project like this is under-planning content before writing code, not coordinating with collaborators. Before any HTML was written, I produced a page-by-page content plan (six pages: a home page, four era pages, and a sources page) and a fact sheet with citations, so that the later wireframes and code had real content to work from rather than placeholder text. I also identified a specific technical risk during planning — that relying entirely on JavaScript to inject a page's only heading could leave the page structurally broken if JavaScript failed, or invisible to tools that don't run it — and planned around it by deciding upfront that headings and primary structure would stay in static HTML, with only supporting content (paragraphs, event cards) coming from JSON.

C. Development process: prototype designs

I produced low-fidelity wireframes, in both desktop and mobile widths, for every distinct page layout on the site: the home page, a shared "era page" template (used by all four era pages, since they share one layout populated from different JSON data), and the sources page. The wireframes deliberately used grey placeholder boxes rather than final colours or copy, so that layout and structure — where the navigation sits, how many columns the era cards use, how the timeline of events is laid out — could be reviewed on their own terms before any visual design was committed to. The mobile wireframes in particular tested how the header navigation should behave on a

narrow screen: rather than hiding links behind a hamburger menu, the design lets the six navigation links wrap onto a second line, keeping every page reachable in one tap rather than two, which mattered later for the accessibility goals in the testing stage.

D. Development process: developing the code

The site is six static HTML pages sharing one stylesheet (`css/style.css`) and one small JavaScript module (`js/render.js`). Each era page stores its hero text and its list of historical events in its own JSON file under `data/`, and the home page pulls a cross-era "highlights" JSON file for its quick-scan timeline. Content is rendered client-side with the Handlebars.js templating engine — vendored locally in `lib/handlebars.min.js` so the site does not depend on a live CDN connection — fetched over HTTP from a local static server, since `fetch()` cannot load local JSON files opened directly from disk.

Before any JSON reaches a template, `js/render.js` runs a hand-written validator (`validateEraData` / `validateTimelineData``) that checks every field a template depends on: that `events`` is a non-empty array, that each event has a `year`` (number), a `headline`` and `detail`` (non-empty strings), and a `sourceName`/`sourceUrl`` (a real `http(s)`` URL) — since a missing or malformed source citation would be a bigger problem on a history site than a missing decoration would be. If validation fails, the page renders a visible error banner listing exactly which fields failed, rather than letting Handlebars throw on unexpected data or silently render blank fields.

I tested this validator directly (not just by hoping the real JSON files happen to be valid): feeding it a deliberately malformed object confirmed it correctly reports every missing or wrong-typed field, and that the error banner renders as intended in the DOM. Illustrations (one hero image per era, plus one for the home page) are original SVGs drawn specifically for this project, so no external image licensing was needed.

E. Testing: code validation reports and actions taken

I ran the W3C Nu Html Checker (`validator.w3.org/nu`) against all six pages before making any fixes. Four era pages both lacked a level-1 heading in their raw HTML and had a `<section>` with no heading — because their entire hero, including the page's only `<h1>`, was being injected by JavaScript after a `fetch()` resolved, so the checker (and any tool or crawler that doesn't run JavaScript) saw an empty hero section. `sources.html` had a heading-level skip (`<h3>` directly under `<h1>`, with no `<h2>` between them). I fixed both: the four era pages now have a real, static `<h1>` and era-label in the HTML from page load (JavaScript fills in its text once the JSON has loaded, rather than creating the element from scratch), and `sources.html`'s four bibliography headings were changed from `<h3>` to `<h2>`. Re-running the checker afterwards confirmed all six pages now validate with zero errors or warnings.

I then ran axe-core 4.10.2 (loaded temporarily via CDN, not shipped with the site) against every page. It found one real, moderate issue — the `<section class="hero">` had no accessible name, so it wasn't recognised as a proper landmark region for screen-reader navigation — fixed by adding `aria-labelledby` on each hero section, pointing at that page's own heading. It also flagged eight "serious" colour-contrast violations on the header/footer navigation text. Investigating this, axe reported the text's background as `#f4f6fb` — a colour that appears nowhere in the header/footer CSS — while `getComputedStyle()` on the actual header element confirmed its real, fully-opaque

background is `#0b1220`. Manually calculating the WCAG contrast ratio (the relative-luminance formula from WCAG 2.1 §1.4.3) for the real colours in use gives 16.7:1 for the site title and 9.08:1 for the nav links against their real background — both comfortably above the 4.5:1 AA threshold, and above the 7:1 AAA threshold too. I documented this as an automated-tool false positive (in `docs/testing-reports/`) rather than "fixing" a colour scheme that was already compliant, since blindly changing already-correct colours to satisfy a tool would not have improved the actual site.

F. Reflections on what I learned

The most useful lesson was that automated tools verify different things than they seem to. The HTML validator's warnings looked cosmetic ("section lacks heading") but pointed at a real architectural problem: content that only exists after JavaScript runs is invisible to anything that doesn't run JavaScript, including some crawlers, and briefly invisible to everyone else too. Fixing it properly meant restructuring how the page and its script divide responsibility for the heading, not just silencing a warning. The accessibility check taught the opposite lesson: automated output still needs to be checked against the real, rendered page rather than trusted at face value — the reported background colour didn't match any colour the CSS could possibly produce, which was itself the clue that the tool, not the design, was wrong. Validating my own JSON-validation code (rather than assuming it worked because it looked correct) was a small habit that paid off directly, since it's the exact thing that would let a future contributor's malformed data fail loudly instead of breaking the page silently.